

Clothing Information Retrieval System

Assignment 1 - Information Retrieval
Project Documentation

*A term-based search engine over a 100-document clothing corpus
implemented from scratch in Python.*

1. Overview

This project implements a complete Information Retrieval (IR) system for a corpus of 100 clothing product documents. Each document describes a single garment with a document ID, category, title, and free-text description. The system indexes the corpus and lets the user retrieve relevant documents using several retrieval models: TF-IDF with cosine similarity, BM25, Boolean set retrieval, Jaccard similarity, and phrase/proximity search through a positional index. Comprehensive evaluation metrics (precision, recall, F1, AP, MAP, NDCG) are included, along with optional query expansion and an interactive command-line interface.

All components - preprocessing, stemming, indexing, and ranking - are implemented from scratch in a single Python module with no third-party dependencies.

2. Corpus

The corpus file corpus_100.txt contains 100 documents in an XML-like format. Each document has four fields:

```
<DOC>
  <DOCID>      D001 ... D100
  <CATEGORY>   T-Shirt | Shirt | Jeans | Kurta | Saree | Dress |
               Hoodie | Jacket | Leggings | Sweatshirt
  <TITLE>      product name (gender, style, garment, colour)
  <TEXT>       fabric, features, sizing and usage description
</DOC>
```

There are exactly 10 documents per category, giving a balanced 10 x 10 corpus.

Corpus Statistics

- Total documents: 100
- Unique terms after stemming and stop-word removal: 122
- Categories (10 each): T-Shirt, Shirt, Jeans, Kurta, Saree, Dress, Hoodie, Jacket, Leggings, Sweatshirt

3. Processing Pipeline

3.1 Preprocessing

Every document's title and text are combined and run through the preprocessing pipeline:

- Lowercasing - all text converted to lowercase.
- Punctuation removal - non-alphanumeric characters stripped.
- Tokenization - text split into individual words.
- Stop-word removal - a list of ~90 common English stop-words removed (the, and, for, with...).
- Short-token removal - single-character tokens dropped.
 - Stemming - a self-contained Porter stemmer reduces words to root forms (checked->check, lavender->lavend).

3.2 Index Construction

Two indexes are built over the preprocessed tokens:

- Inverted index: term -> set of document IDs containing the term. This powers Boolean search and candidate

retrieval for ranked search.

- Positional index: term $\rightarrow$ {doc_id: [positions]}. Stores the exact position of each term in each document so phrase and proximity queries can be answered by merging position lists.

The model also computes IDF (inverse document frequency) for every term and a normalized TF-IDF vector for every document. The average document length is recorded for BM25.

4. Retrieval Models

4.1 TF-IDF with Cosine Similarity

TF-IDF weights each term by how often it appears in a document (TF) and how rare it is across the corpus (IDF). Documents and queries are represented as L2-normalized TF-IDF vectors and ranked by cosine similarity (dot product of normalized vectors). Higher scores mean stronger relevance.

4.2 BM25

BM25 is a probabilistic ranking function. Each query term contributes an IDF score scaled by its term frequency and dampened by document length (parameters $k_1 = 1.5$, $b = 0.75$). It is well suited to documents of varying length and often outperforms plain TF-IDF.

4.3 Boolean Search (AND / OR / NOT)

Classic set-based retrieval over the inverted index. AND returns documents containing all query terms, OR returns documents containing any term, and NOT subtracts documents of later terms from the first. Results are unranked.

4.4 Jaccard Similarity

Treats query and document as sets of terms and scores them by $|\text{intersection}| / |\text{union}|$. A simple overlap measure that ignores term weight, used here as a comparison baseline.

4.5 Phrase and Proximity Search (Positional Index)

Because the positional index records term positions, the system answers two further query types:

- Phrase search (phrase [query]): query terms must appear adjacent and in order. e.g. phrase regular fit matches "Men's Regular Fit Kurta - Pink".
- Proximity search (near [query]): query terms must appear in order within a small window (4 positions). e.g. near winter jacket matches "Women's Quilted Winter Jacket".

Position lists are merged with a pointer-scanning algorithm that walks each term's position list and accepts a document when every query term occurs within the allowed gap in order.

4.6 Query Expansion

Expands a query with related terms that co-occur frequently with the original terms across the corpus, then re-runs the expanded query through TF-IDF. Broadens retrieval when the user is unsure of the exact vocabulary.

5. Evaluation Metrics

Five predefined test queries with gold relevance judgments evaluate the retrieval quality of TF-IDF, BM25, and Jaccard. Correct documents per query are the 10 documents of the target category.

5.1 Precision, Recall, F1

- Precision = (relevant retrieved) / (total retrieved).
- Recall = (relevant retrieved) / (total relevant).
- F1 = $2PR / (P + R)$, the harmonic mean of the two.

5.2 Average Precision (AP) and MAP

AP is computed at every rank where a relevant document appears, making it sensitive to ranking order. MAP averages AP over all test queries. A perfect ordering gives AP = MAP = 1.0.

5.3 NDCG (Normalized Discounted Cumulative Gain)

NDCG values documents by rank, discounting the gain of lower-ranked documents by $\log_2(\text{rank})$. It is normalized by the ideal ranking, so NDCG = 1.0 means the top documents are exactly the relevant ones in the best order.

Results on the test set

TF-IDF and BM25 both achieve perfect scores ($P = R = F1 = AP = MAP = NDCG = 1.0$) because the test queries were built to match their categories exactly. Jaccard scores lower overall (MAP = 0.846, mean NDCG = 0.885); on the first query only 3 of its 10 hits are relevant, giving AP = 0.23 and NDCG = 0.42. This demonstrates that the ranking-aware metrics can differentiate methods with identical precision.

6. Usage

Requirements

- Python 3.8+ (developed and tested on Python 3.13).
- No third-party packages required.

Running the system

```
cd "C:\Users\SHRESTH\OneDrive\Desktop\CSD\IR\Assignments"
python clothing_ir_model.py
(or: py clothing_ir_model.py)
```

The program loads the corpus, builds the indexes, prints corpus statistics, and enters an interactive prompt.

Interactive commands

```
[query]          TF-IDF cosine similarity ranked search
bm25 [query]     BM25 ranked search
jaccard [query]  Jaccard set similarity search
phrase [query]   Exact phrase search (adjacent, in order)
near [query]     Proximity search (in order, within 4 words)
and [query]      Boolean AND      or [query] Boolean OR
not [query]      Boolean NOT      expand [q] Query expansion
cat [name]       List docs in a category
stats           Corpus statistics  history  Query log
eval           Run P/R/F1/AP/MAP/NDCG evaluation on all methods
help           Show commands      quit      Exit
```

7. Programmatic Usage

The model can also be used from Python code without the interactive prompt:

```
from clothing_ir_model import ClothingIRModel

ir = ClothingIRModel()
ir.load_corpus("corpus_100.txt")
ir.build_index()

results = ir.tfidf_search("black cotton shirt", top_k=5)
for doc, score in results:
    print(doc.title, score)
```

8. Project Files

```
clothing_ir_model.py   - the full IR system (indexing, search, CLI)
corpus_100.txt         - the 100-document input corpus
README.md             - markdown documentation
Assignment_Clothing_IR.pdf - this document
```

9. Component Map

PorterStemmer	-> protected English stemmer
Document (dataclass)	-> one indexed document
ClothingIRModel	-> main engine
load_corpus()	parse corpus_100.txt
tokenize()	preprocessing + stemming
build_index()	TF, IDF, inverted, positional index
tfidf_search()	cosine similarity ranking
bm25_search()	BM25 ranking
jaccard_search()	Jaccard ranking
phrase_search()	positional phrase matching
phrase_search_ranked()	phrase ranking with TF-IDF
boolean_search()	AND / OR / NOT
query_expansion()	co-occurrence expansion
average_precision()	AP metric
ndcg()	NDCG metric
evaluate()	P/R/F1/AP/NDCG per query
evaluate_all()	MAP, mean NDCG across queries
interactive_mode()	command-line user interface